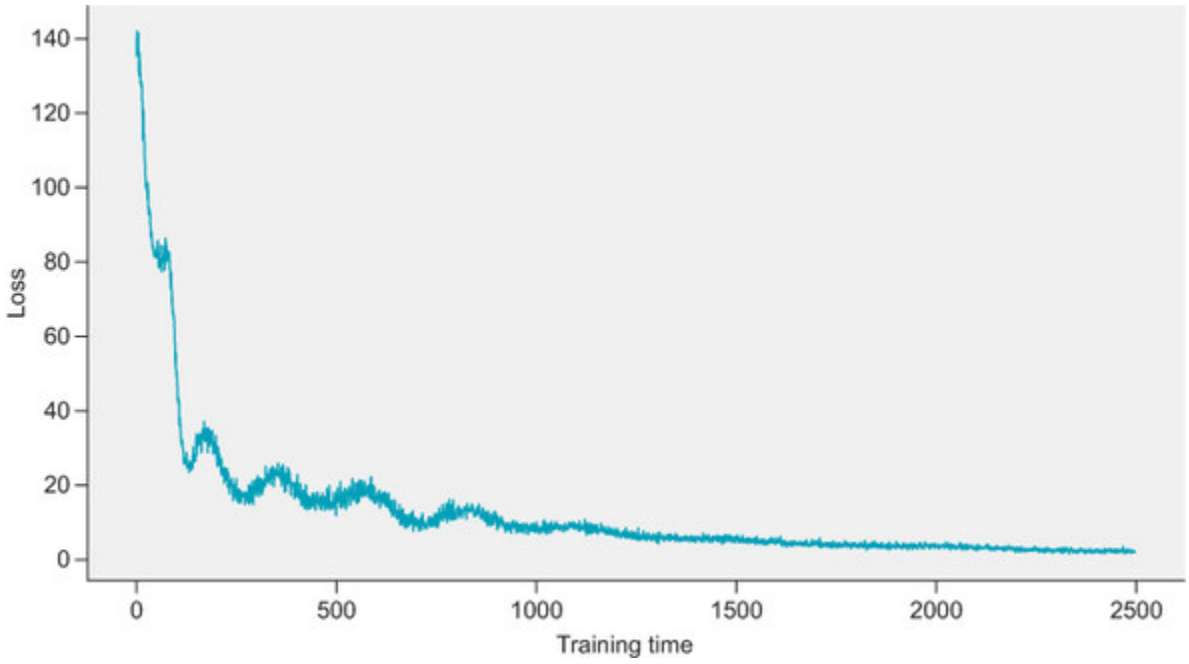
Figure A.3. The loss function for our neural network trained on the MNIST dataset.



We can easily achieve greater than 95% accuracy by improving the training algorithm with a more sophisticated version of gradient descent. In listing A.4 we implemented our own version of *stochastic gradient descent*, the *stochastic* part because we are randomly taking subsets from the dataset and computing gradients based on that, which gives us noisy estimates of the true gradient given the full set of data.

PyTorch includes built-in optimizers, of which stochastic gradient descent (SGD) is one. The most popular alternative is called Adam, which is a more sophisticated version of SGD. We just need to instantiate the optimizer with the model parameters.

Listing A.5. Using the Adam optimizer

```
mnist_data = TV.datasets.MNIST("MNIST", train=True, download=False)

lr = 0.001
epochs = 5000
batch_size = 500
lossfn = torch.nn.CrossEntropyLoss()            1
optim = torch.optim.Adam(params=[w1,w2],lr=lr)  2
for i in range(epochs):
    rid =
```